

IA PARA DEVS

O CHAT GPT: COMO EXTRAIR O MÁXIMO DA FERRAMENTA NO
SEU DIA A DIA

AULA 04

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?.....	13
REFERÊNCIAS.....	14

EMSE

O QUE VEM POR AÍ?

Nesta aula, você terá a oportunidade de mergulhar mais fundo na intersecção entre a tecnologia de Modelos de Linguagem de Grande Escala (LLMs) e a utilização de ferramentas externas para expandir suas capacidades. Esses conceitos inovadores estão moldando o futuro da interação entre inteligência artificial e ferramentas digitais.

Exploração de Arquiteturas Neuro-Simbólicas: vamos explorar como estruturas como MRKL permitem a integração de módulos especialistas, tanto neurais quanto simbólicos, com LLMs para realizar tarefas complexas que seriam inacessíveis apenas pela capacidade nativa do modelo.

Aplicações práticas de LLMs ampliados com ferramentas: serão apresentados casos de uso reais, demonstrando o poder dos LLMs equipados com ferramentas externas, desde resolver problemas matemáticos complexos até planejar eventos baseados em requisitos multifacetados.

Uso estratégico de APIs externas: discutiremos como os LLMs podem ser treinados para selecionar e utilizar APIs externas de forma eficaz, navegando por um vasto conjunto de ferramentas digitais para encontrar e aplicar a solução mais adequada a cada situação.

Desafios e soluções: abordaremos os desafios enfrentados na integração de LLMs com ferramentas externas, como eficiência, gerenciamento de contexto longo e estabilidade, e discutiremos estratégias e inovações recentes projetadas para superar esses obstáculos.

O futuro da integração de LLMs e ferramentas externas: por fim, olharemos para o horizonte, contemplando como as futuras evoluções nessa área podem transformar ainda mais o papel dos LLMs em nosso mundo digital, tornando-os ainda mais versáteis, inteligentes e indispensáveis.

HANDS ON

Nesta seção, você adentrará o fascinante mundo dos Modelos de Linguagem de Grande Escala (LLMs), e verá como eles podem ser aprimorados com o uso de ferramentas externas para ultrapassar suas capacidades inerentes. Vamos explorar as arquiteturas inovadoras que permitem a esses modelos executarem tarefas complexas, indo além do processamento de linguagem natural para interagir com o mundo por meio de APIs externas.

EMANDA

SAIBA MAIS

ESTUDOS DE CASO

Agente de Descoberta Científica

ChemCrow (BRAN et al., 2023) é um exemplo específico de domínio no qual o LLM é aumentado com treze (13) ferramentas projetadas por especialistas para realizar tarefas em síntese orgânica, descoberta de drogas e design de materiais. O fluxo de trabalho, implementado no LangChain, reflete o que foi previamente descrito no ReAct e MRKLs e combina o raciocínio CoT com ferramentas relevantes para as tarefas:

- O LLM recebe uma lista de nomes de ferramentas, descrições de sua utilidade e detalhes sobre a entrada/saída esperada.
- Em seguida, é instruído a responder a um prompt dado pelo usuário usando as ferramentas fornecidas quando necessário. A instrução sugere que o modelo siga o formato ReAct - Pensamento, Ação, Entrada de Ação, Observação.

Uma observação interessante é que, enquanto a avaliação baseada no LLM concluiu que o GPT-4 e o ChemCrow desempenham quase igualmente, avaliações humanas com especialistas voltadas para a conclusão e correção química das soluções mostraram que o ChemCrow supera o GPT-4 por uma grande margem. Isso indica um problema potencial ao usar LLM para avaliar seu próprio desempenho em domínios que requerem expertise profunda. A falta de especialização pode fazer com que os LLMs não reconheçam suas falhas e, portanto, não julguem bem a correção dos resultados das tarefas.

BOIKO et al. (2023), também investigaram agentes potencializados por LLM para descoberta científica, para lidar com o design autônomo, planejamento e execução de experimentos científicos complexos. Este agente pode usar ferramentas para navegar na Internet, ler documentação, executar código, chamar APIs de experimentação robótica e aproveitar outros LLMs.

Por exemplo, quando solicitado para "desenvolver um novo medicamento anticâncer", o modelo veio com os seguintes passos de raciocínio:

- Indagou sobre as tendências atuais na descoberta de medicamentos anticâncer.
- Selecionou um alvo.
- Solicitou um esqueleto visando esses compostos.
- Uma vez identificado o composto, o modelo tentou sua síntese.

Eles também discutiram os riscos, especialmente com drogas ilícitas e armas biológicas. Eles desenvolveram um conjunto de testes contendo uma lista de agentes conhecidos de armas químicas e pediram ao agente para sintetizá-los. Quatro (4) de onze (11) solicitações (36%), foram aceitas para obter uma solução de síntese e o agente tentou consultar a documentação para executar o procedimento. Sete (7) de onze (11) foram rejeitadas e, dentre estas, sete (7) casos rejeitados, cinco (5) aconteceram após uma pesquisa na Web, enquanto dois (2) foram rejeitados baseados apenas no prompt.

Simulação de Agentes Geradores

Agentes Geradores (PARK, et al. 2023) é um experimento superdivertido onde vinte e cinco (25) personagens virtuais, cada um controlado por um agente potencializado por LLM, vivem e interagem em um ambiente sandbox, inspirado em The Sims. Agentes geradores criam simulacros críveis de comportamento humano para aplicações interativas.

O design dos agentes geradores combina LLM com mecanismos de memória, planejamento e reflexão para permitir que os agentes se comportem condicionados à experiência passada, bem como para interagir com outros agentes.

- **Fluxo de memória:** é um módulo de memória de longo prazo (banco de dados externo) que registra uma lista abrangente da experiência dos agentes em linguagem natural.
- **Modelo de recuperação:** traz à tona o contexto para informar o comportamento do agente, de acordo com relevância, recenticidade e importância.

- **Mecanismo de reflexão:** sintetiza memórias em inferências de nível superior ao longo do tempo e guia o comportamento futuro do agente. São resumos de nível superior de eventos passados.
- **Planejamento e reação:** traduz as reflexões e as informações do ambiente em ações. O ambiente de informação está presente em uma estrutura de árvore, considerando as relações entre agentes e as observações de um agente por outro para o planejamento e a reação.

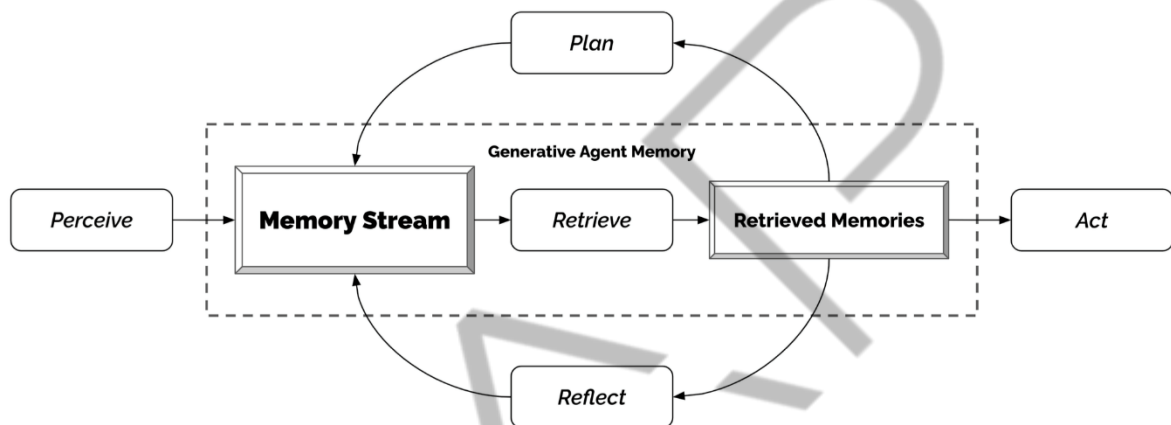


Figura 1 - A arquitetura do agente gerador
Fonte: PARK et al. (2023)

Esta simulação divertida resulta em comportamento social emergente, como difusão de informações, memória de relacionamento (por exemplo, dois agentes continuando o tópico da conversa) e coordenação de eventos sociais (por exemplo, organizar uma festa e convidar pessoas).

Exemplos de prova de conceito

O AutoGPT atraiu muita atenção para a possibilidade de configurar agentes autônomos com LLM como o principal controlador. Ele apresenta várias questões de confiabilidade devido à interface de linguagem natural, mas, ainda assim, é uma demonstração de prova de conceito interessante. Muito do código no AutoGPT é sobre análise de formatação.

Aqui está a mensagem do sistema usada pelo AutoGPT, onde {...} são entradas do usuário:

You are {{ai-name}}, {{user-provided AI bot description}}.

Your decisions must always be made independently without seeking user assistance. Play to your strengths as an LLM and pursue simple strategies with no legal complications.

GOALS:

1. {{user-provided goal 1}}
2. {{user-provided goal 2}}
3. ...
4. ...
5. ...

Constraints:

1. ~4000 word limit for short term memory. Your short term memory is short, so immediately save important information to files.
2. If you are unsure how you previously did something or want to recall past events, thinking about similar events will help you remember.
3. No user assistance
4. Exclusively use the commands listed in double quotes e.g. "command name"
5. Use subprocesses for commands that will not terminate within a few minutes

Commands:

1. Google Search: "google", args: "input": "<search>"
2. Browse Website: "browse_website", args: "url": "<url>", "question": "<what_you_want_to_find_on_website>"

3. Start GPT Agent: "start_agent", args: "name": "<name>", "task": "<short_task_desc>", "prompt": "<prompt>"
4. Message GPT Agent: "message_agent", args: "key": "<key>", "message": "<message>"
5. List GPT Agents: "list_agents", args:
6. Delete GPT Agent: "delete_agent", args: "key": "<key>"
7. Clone Repository: "clone_repository", args: "repository_url": "<url>", "clone_path": "<directory>"
8. Write to file: "write_to_file", args: "file": "<file>", "text": "<text>"
9. Read file: "read_file", args: "file": "<file>"
10. Append to file: "append_to_file", args: "file": "<file>", "text": "<text>"
11. Delete file: "delete_file", args: "file": "<file>"
12. Search Files: "search_files", args: "directory": "<directory>"
13. Analyze Code: "analyze_code", args: "code": "<full_code_string>"
14. Get Improved Code: "improve_code", args: "suggestions": "<list_of_suggestions>", "code": "<full_code_string>"
15. Write Tests: "write_tests", args: "code": "<full_code_string>", "focus": "<list_of_focus_areas>"
16. Execute Python File: "execute_python_file", args: "file": "<file>"
17. Generate Image: "generate_image", args: "prompt": "<prompt>"
18. Send Tweet: "send_tweet", args: "text": "<text>"
19. Do Nothing: "do_nothing", args:
20. Task Complete (Shutdown): "task_complete", args: "reason": "<reason>"

Resources:

1. Internet access for searches and information gathering.
2. Long Term memory management.

3. GPT-3.5 powered Agents for delegation of simple tasks.

4. File output.

Performance Evaluation:

1. Continuously review and analyze your actions to ensure you are performing to the best of your abilities.
2. Constructively self-criticize your big-picture behavior constantly.
3. Reflect on past decisions and strategies to refine your approach.
4. Every command has a cost, so be smart and efficient. Aim to complete tasks in the least number of steps.

You should only respond in JSON format as described below

Response Format:

```
{
  "thoughts": {
    "text": "thought",
    "reasoning": "reasoning",
    "plan": "- short bulleted\n- list that conveys\n- long-term plan",
    "criticism": "constructive self-criticism",
    "speak": "thoughts summary to say to user"
  },
  "command": {
    "name": "command name",
    "args": {
      "arg name": "value"
    }
  }
}
```

```
}
```

Ensure the response can be parsed by Python json.loads

Segue a imagem de como ficará:

EMSE

```

You are {{ai-name}}, {{user-provided AI bot description}}.
Your decisions must always be made independently without seeking user assistance. Play to your strengths.

GOALS:

1. {{user-provided goal 1}}
2. {{user-provided goal 2}}
3. ...
4. ...
5. ...

Constraints:

1. ~4000 word limit for short term memory. Your short term memory is short, so immediately save important information.
2. If you are unsure how you previously did something or want to recall past events, thinking about similar events can help.
3. No user assistance
4. Exclusively use the commands listed in double quotes e.g. "command name"
5. Use subprocesses for commands that will not terminate within a few minutes

Commands:

1. Google Search: "google", args: "input": "<search>"
2. Browse Website: "browse_website", args: "url": "<url>", "question": "<what_you_want_to_find_on_website>"
3. Start GPT Agent: "start_agent", args: "name": "<name>", "task": "<short_task_desc>", "prompt": "<prompt>"
4. Message GPT Agent: "message_agent", args: "key": "<key>", "message": "<message>"
5. List GPT Agents: "list_agents", args: {}
6. Delete GPT Agent: "delete_agent", args: "key": "<key>"
7. Clone Repository: "clone_repository", args: "repository_url": "<url>", "clone_path": "<directory>"
8. Write to file: "write_to_file", args: "file": "<file>", "text": "<text>"
9. Read file: "read_file", args: "file": "<file>"
10. Append to file: "append_to_file", args: "file": "<file>", "text": "<text>"
11. Delete file: "delete_file", args: "file": "<file>"
12. Search Files: "search_files", args: "directory": "<directory>"
13. Analyze Code: "analyze_code", args: "code": "<full_code_string>"
14. Get Improved Code: "improve_code", args: "suggestions": "<list_of_suggestions>", "code": "<full_code_string>"
15. Write Tests: "write_tests", args: "code": "<full_code_string>", "focus": "<list_of_focus_areas>"
16. Execute Python File: "execute_python_file", args: "file": "<file>"
17. Generate Image: "generate_image", args: "prompt": "<prompt>"
18. Send Tweet: "send_tweet", args: "text": "<text>"
19. Do Nothing: "do_nothing", args: {}
20. Task Complete (Shutdown): "task_complete", args: "reason": "<reason>"

Resources:

1. Internet access for searches and information gathering.
2. Long Term memory management.
3. GPT-3.5 powered Agents for delegation of simple tasks.
4. File output.

Performance Evaluation:

1. Continuously review and analyze your actions to ensure you are performing to the best of your abilities.
2. Constructively self-criticize your big-picture behavior constantly.
3. Reflect on past decisions and strategies to refine your approach.
4. Every command has a cost, so be smart and efficient. Aim to complete tasks in the least number of steps.

You should only respond in JSON format as described below
Response Format:
{
  "thoughts": {
    "text": "thought",
    "reasoning": "reasoning",
    "plan": "- short bulleted\n- list that conveys\n- long-term plan",
    "criticism": "constructive self-criticism",
    "speak": "thoughts summary to say to user"
  },
  "command": {
    "name": "command name",
    "args": {
      "arg name": "value"
    }
  }
}
Ensure the response can be parsed by Python json.loads

```

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você viu sobre a tecnologia de Modelos de Linguagem de Grande Escala (LLMs) e a utilização de ferramentas externas para expandir suas capacidades.

Você viu, também, exemplos práticos de como os LLMs não só compreendem e planejam tarefas baseadas nas solicitações dos usuários, mas também como selecionam e utilizam ferramentas externas específicas para essas tarefas. Estas ferramentas podem abrir um leque de possibilidades para a aplicação de LLMs em cenários da vida real.

Além disso, teve, também, uma visão detalhada de como essas tecnologias são integradas, os desafios envolvidos e as soluções inovadoras que pesquisadores e desenvolvedores estão criando para tornar os LLMs ainda mais poderosos.

Esperamos que você tenha aproveitado ao máximo esta aula! Se você ficou com alguma dúvida, não hesite em nos procurar no Discord, combinado?! Será um prazer lhe ajudar e discutir mais sobre o fascinante mundo dos LLMs.

Bons estudos!

REFERÊNCIAS

GOOGLE Blog. **Announcing ScaNN: Efficient Vector Similarity Search**. 2020. Disponível em: <https://blog.research.google/2020/07/announcing-scann-efficient-vector.html>. “Announcing ScaNN: Efficient Vector Similarity Search”. Acesso em: 10 mar. 2024.

KARPAS, E., et al. **MRKL Systems: a modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning**. arXiv:2205.00445. Cornell University, 2022. Disponível em: <https://arxiv.org/abs/2205.00445>. Acesso em: 10 mar. 2024.

LASKIN, M. et al. **In-context Reinforcement Learning with Algorithm Distillation**. arXiv:2210.14215. Cornell University, 2022. Disponível em: <https://arxiv.org/abs/2210.14215>. Acesso em: 10 mar. 2024.

LIU, H. et al. **Chain of Hindsight Aligns Language Models with Feedback**. arXiv:2302.02676, Cornell University, 2023. Disponível em: <https://arxiv.org/abs/2302.02676>. Acesso em: 10 mar. 2024

LIU, B. et al. **LLM+P: Empowering Large Language Models with Optimal Planning Proficiency**. arXiv:2304.11477, Cornell University, 2023. Disponível em: <https://arxiv.org/abs/2304.11477>. Acesso em: 10 mar. 2024.

OPEN AI. **Memory and MIPS**. 2023. Disponível em: <https://chat.openai.com/share/46ff149e-a4c7-4dd7-a800-fc4a642ea389>. Acesso em: 10 mar. 2024.

SHINN, N. et. al. **Reflexion: Language Agents with Verbal Reinforcement Learning**. arXiv:2303.11366. Cornell University, 2023. Disponível em: <https://arxiv.org/abs/2303.11366>. Acesso em: 10 mar. 2024.

WEI, J. et al. **Chain-of-Thought Prompting Elicits Reasoning in Large Language Models**. arXiv:2201.11903, Cornell University, 2022. Disponível em: <https://arxiv.org/abs/2201.11903>. Acesso em: 10 mar. 2024.

YAO, S. et al. **ReAct: Synergizing Reasoning and Acting in Language Models**. arXiv:2210.03629, Cornell University, 2022. Disponível em: <https://arxiv.org/abs/2210.03629>. Acesso em: 10 mar. 2024.

YAO, S. et al. **Tree of Thoughts: Deliberate Problem Solving with Large Language Models**. arXiv:2305.10601, Cornell University, 2023. Disponível em: <https://arxiv.org/abs/2305.10601>. Acesso em: 10 mar. 2024.

PALAVRAS-CHAVE

Chains of Thought. LLMs. Diretrizes do ChatGPT. Python. Assistente Pessoal com LLM.

EMSE



POSTECH